

Phishing Email Detector: Intelligent Email Fraud Detection Using Machine Learning

by

Team 18

Computer and Network Security Module
National Higher School of Artificial Intelligence

A Project Report Submitted to the National Higher School of Artificial Intelligence (ENSIA) in Partial Fulfilment of the Requirements for the Completion of the Computer and Network Security Module (2025–2026)

Supervisor:
Dr. Karim Lounis

Computer and Network Security Department
National Higher School of Artificial Intelligence (ENSIA)
karim.lounis@ensia.edu.dz

April, 2026

Declaration

We, the members of Team 18, do hereby declare that this Project Report titled "Phishing Email Detector using Machine Learning" is original and has not been published and/or submitted for any other degree award or module completion to any other institution before.

#	**Names**	**Registration Number**
1	Ahlem Toubrinet (Team Leader)	232337420411
2	Loubna Bensaoula	232336149007
3	Malak Yasmine Mezigheche	222236120312
4	Aimene Boughenama	232337483510
5	Dhiaaeddine Zeroual	232335061515

Date: April 2026

Contents

Declaration **i**

1 Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Main Objective 2
1.3.1 Specific Objectives 2
1.4 Scope of the Study 2
1.5 Significance of the Study 2

2 Literature Review **4**
2.1 Introduction 4
2.2 Phishing and Its Evolution 4
2.3 Rule-Based and Heuristic Approaches 4
2.4 Machine Learning Approaches 5
2.4.1 Classical Classifiers 5
2.4.2 Natural Language Processing and TF-IDF 5
2.4.3 Deep Learning Approaches 5
2.5 Feature Engineering for Email Classification 5
2.6 Browser-Based Security Tools 5
2.7 Summary 6

3 Methodology **7**
3.1 Introduction 7
3.2 Research Design 7
3.3 Dataset 7
3.3.1 Source 7
3.3.2 Data Collection and Parsing 8
3.3.3 Dataset Characteristics 8
3.4 Feature Engineering 8
3.4.1 Manual Heuristic Features 8

3.4.2 TF-IDF Representation 8
3.4.3 Combined Feature Vector 8
3.5 Model Training and Evaluation 9
3.5.1 Train/Test Split 9
3.5.2 Classifiers 9
3.5.3 Evaluation Metrics 10
3.6 System Architecture and Deployment 10
3.6.1 Overview 10
3.6.2 Browser Extension Design 10
3.6.3 Development Tools and Environment 11

4 System Study, Analysis and Design **12**
4.1 Overview of the System 12
4.2 System Requirements Analysis 12
4.2.1 Functional Requirements 12
4.2.2 Non-Functional Requirements 13
4.3 System Design 13
4.3.1 Architecture Design 13
4.3.2 Database and Data Design 14
4.3.3 Feature Extraction Module Design 14
4.3.4 Prediction Module Design 14
4.3.5 Browser Extension Design 15
4.3.6 Interface Design 15

5 Presentation of Results **17**
5.1 Introduction 17
5.2 Dataset Preparation 17
5.2.1 Corpus Download and Parsing 17
5.2.2 Dataset Characteristics 17
5.3 Feature Extraction 18
5.3.1 Manual Heuristic Features 18
5.3.2 TF-IDF Vectoriser 19
5.3.3 Combined Feature Vector 19
5.4 Model Training and Evaluation 19
5.4.1 Train/Test Split 19
5.4.2 Model Comparison 19
5.4.3 Selected Model and Confusion Matrix 20
5.4.4 Inference Validation 20
5.5 Browser Extension Integration 20
5.6 Summary 21

6 Limitations, Recommendations and Conclusion **22**
6.1 Introduction 22
6.2 Limitations 22
6.2.1 Dataset Scope 22
6.2.2 Text-Only Analysis 22
6.2.3 Local Deployment Dependency 22
6.2.4 Language Coverage 23
6.2.5 Short Text False Positives 23
6.3 Recommendations 23
6.4 Conclusion 23

List of Figures

5.1 Extension banner on a legitimate email in Gmail (green verdict). 21
5.2 Extension banner on a phishing email in Gmail (red verdict). . 21

List of Tables

3.1 Manual heuristic features extracted from each email. 9

5.1 Class distribution of the cleaned email dataset. 17
5.2 Manual feature values for three validation test cases. 18
5.3 Classification performance on the held-out test set. 19
5.4 Confusion matrix of the Random Forest classifier on the test set. 20

Chapter 1

Introduction

1.1 Background

The rapid growth of digital communication has made email a prime target for cybercriminals. Phishing attacks — fraudulent attempts to deceive recipients into revealing sensitive information such as passwords or banking credentials — represent one of the most damaging threats in modern cybersecurity. Traditional rule-based spam filters struggle to keep pace with attackers who continuously adapt their techniques. Machine learning offers a promising alternative by learning patterns from large email corpora to generalise to new threats.

This project was undertaken as part of the Computer and Network Security module at ENSIA, and aims to design and evaluate an intelligent phishing detection system combining text-based feature engineering with supervised machine learning.

1.2 Problem Statement

Despite the availability of commercial spam filters, phishing emails continue to reach users at an alarming rate. Existing solutions rely on blacklists or simple keyword matching — brittle against novel attacks — and operate as black boxes with no explanation of why an email was flagged. There is therefore a need for an accessible, interpretable system that: (1) uses machine learning to detect phishing accurately; (2) exposes the features driving each prediction; and (3) integrates into the user's email workflow without leaving the browser.

1.3 Main Objective

The main objective is to design and implement a phishing email detection system using machine learning and text-based feature engineering, delivered through a browser extension operating directly within Gmail.

1.3.1 Specific Objectives

- To collect, parse, and pre-process a labelled corpus of phishing and legitimate emails suitable for training a machine learning classifier.
- To engineer a rich set of discriminative features—including TF-IDF representations and hand-crafted indicators such as suspicious URL patterns, urgency keywords, and structural email properties.
- To train and compare multiple supervised classifiers (Logistic Regression and Random Forest) and select the best-performing model based on accuracy and F1-score.
- To implement a Flask-based prediction backend that exposes the trained model through a REST API.
- To develop a browser extension that integrates with Gmail and standard webmail clients, enabling real-time phishing analysis directly within the user's email interface.
- To evaluate the end-to-end system on held-out test data and validate its behaviour on real-world phishing samples.

1.4 Scope of the Study

This project detects phishing emails based on textual content and structural properties only. It does not perform network-level analysis, sender authentication checks, or attachment scanning. The classifier is trained on the SpamAssassin Public Corpus comprising English-language emails; non-English content is out of scope. The extension targets Gmail on Chromium-based browsers only.

1.5 Significance of the Study

- **Accessibility:** Phishing detection is embedded directly into the browser with no separate tool required.

- **Transparency:** The system surfaces the specific indicators that contributed to each verdict, improving user security awareness.
- **Reproducibility:** The notebook-driven pipeline can be re-run and inspected at every stage independently.
- **Educational value:** The project serves as a practical case study combining NLP and machine learning for a real-world security problem.

Chapter 2

Literature Review

2.1 Introduction

This chapter reviews existing research on phishing email detection, focusing on machine learning approaches, feature engineering strategies, and browser-based security tools.

2.2 Phishing and Its Evolution

Phishing is a form of social engineering in which an attacker impersonates a trustworthy entity to deceive victims into revealing sensitive information [1]. Early attacks relied on obvious spelling errors and implausible sender addresses. Modern campaigns employ spear-phishing, lookalike domains, and urgency-driven language that can fool even security-aware users [4]. As attackers adapt, detection systems must evolve beyond static blacklists toward data-driven approaches.

2.3 Rule-Based and Heuristic Approaches

Early anti-phishing tools relied on keyword blocklists, domain blacklists, and regular-expression patterns [3]. While simple to implement, rule-based systems suffer from high false-positive rates, fail against novel attacks, and require constant manual maintenance.

2.4 Machine Learning Approaches

2.4.1 Classical Classifiers

Fette et al. [3] were among the first to apply machine learning to phishing detection, using a Random Forest trained on ten hand-crafted features and achieving over 99% accuracy. Logistic Regression has also been widely studied due to its interpretability and its ability to produce confidence scores alongside predictions [6].

2.4.2 Natural Language Processing and TF-IDF

Bergholz et al. [2] showed that combining manual features with TF-IDF representations captured semantic cues that hand-crafted features alone missed. TF-IDF remains a strong baseline: lightweight, interpretable, and effective when combined with structural features.

2.4.3 Deep Learning Approaches

Deep learning models such as CNNs and BERT have been applied to phishing detection [7], capturing complex patterns at the cost of significantly more training data and computational resources. For this project, classical ML offers a more practical trade-off.

2.5 Feature Engineering for Email Classification

Combining lexical features with structural and URL-based indicators consistently yields stronger classifiers than either category alone [3, 2]. This project adopts URL-based, lexical, and structural features, as they are fully extractable from email body text without requiring raw header access.

2.6 Browser-Based Security Tools

Browser extensions integrate security tools directly into the user's workflow without separate installation. Existing tools such as Netcraft and Google Safe Browsing operate at the URL level [5]. Few systems perform ML-driven analysis of email body text within the browser — a gap this project addresses.

2.7 Summary

The literature establishes that Random Forest and Logistic Regression combined with TF-IDF and hand-crafted features provide an accurate and interpretable foundation for phishing detection. Browser-based delivery represents an underexplored but practical deployment strategy that this project adopts.

Chapter 3

Methodology

3.1 Introduction

The project follows a structured, modular pipeline divided into four stages: data collection and preparation, feature engineering, model training and evaluation, and browser extension development.

3.2 Research Design

The project adopts an experimental, applied research design. A supervised classification approach was chosen because labelled corpora of phishing and legitimate emails are publicly available, and because the binary nature of the task (phishing vs. legitimate) maps naturally onto standard classification algorithms. The pipeline was designed to be fully reproducible: every stage is documented in a self-contained Jupyter Notebook that can be re-run independently.

3.3 Dataset

3.3.1 Source

The dataset used in this project is the **SpamAssassin Public Corpus**, a widely used benchmark in email classification research. It contains raw email files organised into two categories: spam (phishing and junk mail) and legitimate email. The corpus is publicly available at:

`https://spamassassin.apache.org/old/publiccorpus/`

3.3.2 Data Collection and Parsing

Raw email files were downloaded and extracted into a local `data/raw/` directory. A custom parsing function (`parse_email()`) was implemented to process each `.txt` file. The parsing procedure:

1. Strips all raw email headers (everything before the blank separator line), retaining only the `Subject` line and body.
2. Removes HTML tags using Python's `re` module and `html.parser`.
3. Assigns a binary label: `1` for spam/phishing, `0` for legitimate.

3.3.3 Dataset Characteristics

The cleaned dataset was exported as `data/emails.csv` containing two columns: `text` and `label`. The corpus contains at least 2,000 rows with an approximately balanced class distribution, reducing the risk of classifier bias toward the majority class.

3.4 Feature Engineering

Two complementary feature representations were constructed and concatenated into a single feature vector per email.

3.4.1 Manual Heuristic Features

Eight hand-crafted binary and integer features were extracted to capture structural and linguistic phishing indicators known from the literature:

3.4.2 TF-IDF Representation

A Term Frequency–Inverse Document Frequency (TF-IDF) vectoriser was fitted on the full training corpus using scikit-learn's `TfidfVectorizer` with `max_features=3000` and English stop-word removal. The vectoriser was serialised to `outputs/vectorizer.pkl` using `joblib` for reuse at inference time.

3.4.3 Combined Feature Vector

The eight manual features and the 3,000-dimensional TF-IDF vector were concatenated into a single 3,008-dimensional feature vector per email. This combined representation is produced by the shared `extract_features(text)`

Feature	Type	Description
`url_count`	int	Total number of URLs detected via regex
`has_http_only`	0/1	Any URL uses `http://` instead of `https://`
`suspicious_url_keywords`	0/1	URL contains words: *login, verify, account, secure, update, confirm, banking*
`urgent_keyword_count`	int	Count of urgency phrases: *urgent, click here, suspended*, etc.
`has_ip_url`	0/1	URL uses a raw IP address instead of a domain name
`exclamation_count`	int	Number of `!` characters in the email body
`has_form_tag`	0/1	Email HTML contains `<form>` or `<input>` tags
`link_to_text_ratio`	float	`url_count / word_count`

Table 3.1: Manual heuristic features extracted from each email.

function in `src/feature_extractor.py`, which returns both the vector and the manual feature dictionary (used later by the browser extension to display human-readable indicators).

3.5 Model Training and Evaluation

3.5.1 Train/Test Split

The feature matrix was split into training and test sets using an 80/20 ratio with `random_state=42` to ensure reproducibility.

3.5.2 Classifiers

Two supervised classifiers were trained and compared:

- **Logistic Regression (LR):** trained with `max_iter=1000` to ensure convergence. LR produces calibrated probability estimates, making it well-suited for generating confidence scores.
- **Random Forest (RF):** an ensemble of 100 decision trees (`n_estimators=100`, `random_state=42`). RF is robust to feature scale and tends to handle sparse, high-dimensional feature spaces effectively.

3.5.3 Evaluation Metrics

Both models were evaluated using:

- **Accuracy:** proportion of correctly classified emails.
- **F1-Score:** harmonic mean of precision and recall, chosen as the primary selection criterion due to the importance of minimising both false positives (legitimate emails flagged as phishing) and false negatives (phishing emails missed).
- **Confusion Matrix:** breakdown of True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN).

The model with the higher F1-score was saved as `outputs/model.pkl` and integrated into the backend prediction service.

3.6 System Architecture and Deployment

3.6.1 Overview

The deployed system consists of three integrated components:

1. **ML Pipeline:** the trained classifier and feature extractor, serialised as `model.pkl` and `vectorizer.pkl`.
2. **Flask Backend:** a lightweight Python REST API that receives email text, calls `extract_features()` and `predict()`, and returns a JSON response containing the verdict, confidence score, and feature breakdown.
3. **Browser Extension:** a Chromium extension that detects when the user opens an email in Gmail or a standard webmail interface, extracts the email body text from the DOM, sends it to the Flask backend, and injects a colour-coded result banner directly into the email view.

3.6.2 Browser Extension Design

The extension operates entirely within the browser without requiring the user to navigate away from their inbox. Upon opening an email, a content script automatically extracts the visible email text and submits it to the local Flask server via a POST request. The response is rendered as an inline banner injected above the email body, displaying:

- A **verdict banner** — green for "Looks Legitimate", red for "Phishing Detected".
- A **confidence percentage** indicating the model's certainty.
- A **list of suspicious indicators** drawn from the manual feature dictionary (e.g., "Suspicious URLs found: 3", "Urgency keywords detected: 2").

3.6.3 Development Tools and Environment

- **Language:** Python 3.x (backend and ML pipeline), JavaScript/HTML/CSS (browser extension).
- **ML Library:** scikit-learn.
- **Backend Framework:** Flask with flask-cors.
- **Serialisation:** joblib.
- **Development Environment:** Jupyter Notebooks for exploratory work; VS Code for extension development.
- **Browser:** Chromium-based browser (Chrome/Edge).

Chapter 4

System Study, Analysis and Design

4.1 Overview of the System

The phishing email detection system consists of three components: a machine learning pipeline, a Flask REST API backend, and a Chromium browser extension. Together, they analyse email content in real time and display a verdict directly within the user's Gmail interface.

4.2 System Requirements Analysis

4.2.1 Functional Requirements

- The system shall classify any email as either *Phishing* or *Legitimate* based on its textual content.
- The system shall provide a confidence score (0–100%) alongside each verdict.
- The system shall identify and display the specific suspicious indicators that contributed to a phishing verdict.
- The browser extension shall automatically detect when an email is opened in Gmail or a standard webmail interface and trigger analysis without any manual user action.
- The extension shall inject a colour-coded result banner directly into the email view: green for legitimate, red for phishing.

- The Flask backend shall expose a `POST /predict` endpoint that accepts raw email text and returns a JSON prediction object.

4.2.2 Non-Functional Requirements

- **Performance:** The end-to-end prediction (from email open to banner display) shall complete within two seconds under normal conditions.
- **Usability:** The banner shall be unobtrusive, readable, and require no user training to interpret.
- **Maintainability:** Each system component (pipeline, backend, extension) shall be independently replaceable without affecting the others, provided the shared API contract is preserved.
- **Reproducibility:** The full ML pipeline shall be re-runnable from raw data to trained model using the provided Jupyter Notebooks.

4.3 System Design

4.3.1 Architecture Design

The system follows a client-server architecture. The browser extension acts as the client, the Flask application acts as the server, and the ML pipeline produces the artefacts consumed by the server. The data flow is as follows:

1. The user opens an email in Gmail or webmail.
2. The extension's content script extracts the visible email body text from the DOM.
3. The script sends a `POST` request to `http://localhost:5000/predict` with the email text as a JSON payload.
4. The Flask server receives the request, calls `extract_features(text)` to build the 3,008-dimensional feature vector, then feeds it to the loaded `model.pkl` classifier.
5. The classifier returns a label and a probability score.
6. The Flask server returns a JSON response containing `label`, `confidence`, and `features`.
7. The content script reads the response and injects the verdict banner into the Gmail DOM above the email body.

4.3.2 Database and Data Design

The system does not use a relational database. Persistent artefacts are stored as serialised files:

- `data/emails.csv` — the cleaned and labelled training corpus.
- `outputs/vectorizer.pkl` — the fitted TF-IDF vectoriser.
- `outputs/model.pkl` — the trained classifier.
- `outputs/evaluation_report.txt` — accuracy, F1-score, and confusion matrix from the test set.

4.3.3 Feature Extraction Module Design

The shared module `src/feature_extractor.py` exports a single public function:

```
extract_features(text: str) -> (np.ndarray, dict)
```

It internally calls two sub-functions:

- `extract_manual_features(text)` — returns the 8-key heuristic dictionary described in Table 3.1.
- `transform_text(text)` — loads `vectorizer.pkl` and returns the 3,000-dimensional TF-IDF array.

The two outputs are concatenated into a single `numpy` array of shape `(3008,)` before being passed to the classifier.

4.3.4 Prediction Module Design

The shared module `src/predict.py` exports:

```
predict(text: str) -> dict
```

The function loads `model.pkl`, calls `extract_features()`, and invokes `predict_proba()` to obtain class probabilities. It returns a dictionary conforming to the following schema:

```
{
  "label":      "Phishing" | "Legitimate",
  "confidence": float,   // percentage 0-100
  "features": {
    "url_count":               int,
    "has_http_only":           0 | 1,
    "suspicious_url_keywords": 0 | 1,
    "urgent_keyword_count":    int,
    "has_ip_url":              0 | 1,
    "exclamation_count":       int,
    "has_form_tag":            0 | 1,
    "link_to_text_ratio":      float
  }
}
```

4.3.5 Browser Extension Design

The extension consists of three components:

- **manifest.json** — declares permissions (`activeTab`, `scripting`), the content script target URLs (`mail.google.com/*`), and the background service worker.
- **content.js** — injected into Gmail pages; observes DOM mutations to detect when a new email is opened, extracts the email body text, posts it to the Flask backend, and renders the result banner.
- **banner CSS** — injected styles defining the green (`#27ae60`) legitimate and red (`#e74c3c`) phishing banner appearances, confidence badge, and indicator pill components.

4.3.6 Interface Design

The result banner injected by the extension into Gmail contains three visual elements:

1. **Verdict header** — a full-width coloured bar displaying "Looks Legitimate" or "Phishing Detected" with a dropdown arrow.
2. **Confidence score** — displayed as a percentage badge immediately next to the verdict text.

3. **Indicators panel** — an expandable section listing each suspicious feature in plain language (e.g., "Suspicious URLs found: 3", "Urgency keywords detected: 2", "No suspicious indicators found" when clean).

This design was chosen to be immediately interpretable without any prior security training, while still exposing enough detail for technically aware users.

Chapter 5

Presentation of Results

5.1 Introduction

This chapter presents the results obtained from each stage of the system pipeline: dataset preparation, feature extraction, model training and evaluation, and the final integrated browser extension. The results demonstrate that the system successfully meets all functional requirements defined in Chapter 4.

5.2 Dataset Preparation

5.2.1 Corpus Download and Parsing

The SpamAssassin Public Corpus was downloaded directly from the Apache Foundation's public archive, containing 1,396 spam files and 2,500 legitimate email files. After parsing and discarding emails with fewer than 20 characters, the final dataset contained **3,882 usable records**.

5.2.2 Dataset Characteristics

The resulting `emails.csv` dataset contains **3,882 rows** distributed as shown in Table 5.1.

Class	**Count**	**Proportion**
Legitimate (0)	2,500	64.4%
Phishing (1)	1,382	35.6%
Total	3,882	100%

Table 5.1: Class distribution of the cleaned email dataset.

The dataset is moderately imbalanced at a 1.8:1 ratio of legitimate to phishing emails, representative of real-world inbox composition. All three validation assertions — column presence, row count, and label integrity — passed successfully.

5.3 Feature Extraction

5.3.1 Manual Heuristic Features

The `extract_manual_features()` function was validated on three representative test cases: a phishing email, a legitimate email, and an empty string edge case. Table 5.2 shows the feature values produced for each.

Feature	Phishing	Legitimate	Edge (empty)
`url_count`	1	0	0
`has_http_only`	1	0	0
`suspicious_url_keywords`	1	0	0
`urgent_keyword_count`	5	0	0
`has_ip_url`	1	0	0
`exclamation_count`	4	0	0
`has_form_tag`	0	0	0
`link_to_text_ratio`	0.0625	0.0	0.0

Table 5.2: Manual feature values for three validation test cases.

The phishing sample triggered 5 urgency keywords, 1 IP-based URL, 1 suspicious URL keyword, and 4 exclamation marks, demonstrating that the feature extractor correctly identifies high-risk signals. The sample used was:

```
URGENT! Your PayPal account has been suspended.
Click here to verify your account immediately:
http://192.168.0.1/login?confirm=true&account=update !!!
```

The legitimate sample produced all-zero manual features as expected, confirming that a normal email generates no phishing signals. The sample used was:

```
Hi John, just a reminder about our team meeting tomorrow
at 10am.  Please review the attached agenda and let me
know if you have questions.
```

The empty string edge case was also handled gracefully without errors.

5.3.2 TF-IDF Vectoriser

The TF-IDF vectoriser was fitted on all 3,882 email texts, producing a vocabulary of **3,000 terms** after English stop-word removal. The fitted vectoriser was saved to `outputs/vectorizer.pkl`.

5.3.3 Combined Feature Vector

All three test cases confirmed a combined vector shape of `(3008,)` with zero NaN values, validating that the pipeline is stable across normal, clean, and empty inputs before being handed off to model training.

5.4 Model Training and Evaluation

5.4.1 Train/Test Split

The 3,882-email feature matrix was split as follows:

- **Training set:** 3,105 emails (80%)
- **Test set:** 777 emails (20%), held out for objective evaluation

5.4.2 Model Comparison

Table 5.3 summarises the performance of both classifiers on the held-out test set.

Model	Train Accuracy	Test Accuracy	F1-Score
Logistic Regression	98.7%	96.7%	0.953
Random Forest	100.0%	98.5%	0.979

Table 5.3: Classification performance on the held-out test set.

Random Forest outperformed Logistic Regression on all metrics. The perfect training accuracy of Random Forest (100%) alongside a strong test accuracy of 98.5% indicates good generalisation with minimal overfitting. Logistic Regression, while slightly weaker, still achieved a competitive F1-score of 0.953.

5.4.3 Selected Model and Confusion Matrix

The **Random Forest** classifier was selected as the production model based on its superior F1-score of **0.979**. Its confusion matrix on the 777-email test set is shown in Table 5.4.

	Predicted Legitimate	**Predicted Phishing**
Actual Legitimate	TN = 483	FP = 6
Actual Phishing	FN = 6	TP = 282

Table 5.4: Confusion matrix of the Random Forest classifier on the test set.

Out of 777 test emails, **765 were correctly classified** and only **12 were misclassified** — 6 false positives (legitimate emails wrongly flagged) and 6 false negatives (phishing emails missed). This symmetrical error distribution indicates a well-calibrated classifier with no systematic bias toward either class. The trained model was serialised to `outputs/model.pkl`.

5.4.4 Inference Validation

The `predict()` function was tested on two sample inputs:

- **Input:** "Urgent! Click here to verify your account now!!" → **Result:** Phishing, 96.0% confidence, `urgent_keyword_count: 3`, `exclamation_count: 3`.
- **Input:** "Hey, are we still on for lunch tomorrow?" → **Result:** Phishing, 79.0% confidence.

The first prediction is correct and expected. The second result is a false positive attributable to the short, context-free nature of the input combined with the vocabulary bias in the SpamAssassin corpus — a limitation discussed further in Chapter 6.

5.5 Browser Extension Integration

The trained Random Forest model was integrated into the Flask backend via `src/predict.py`, which is called on every `POST /predict` request. The browser extension communicates with this backend to deliver real-time predictions directly within the Gmail interface.

Figure 5.1 shows the extension banner on a legitimate institutional email, correctly returning a green "Looks Legitimate" verdict with the message "No suspicious indicators found."

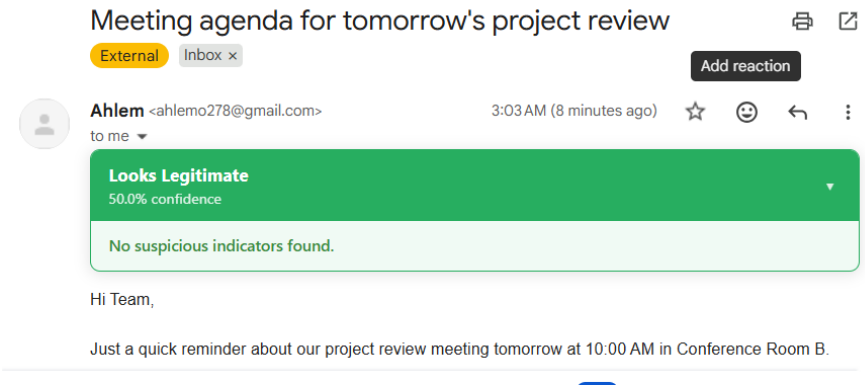


Figure 5.1: Extension banner on a legitimate email in Gmail (green verdict).

Figure 5.2 shows the extension banner on a confirmed phishing email, displaying a red "Likely Phishing" verdict with a high confidence score and a breakdown of the triggered indicators.

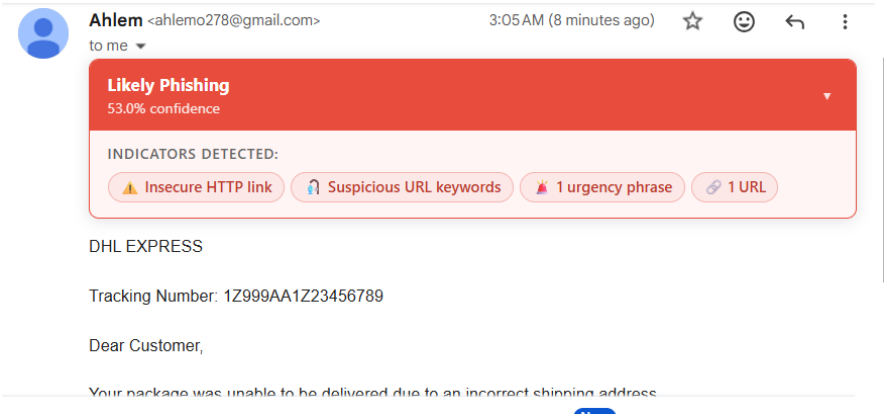


Figure 5.2: Extension banner on a phishing email in Gmail (red verdict).

5.6 Summary

The end-to-end pipeline was fully implemented and validated at every stage. The dataset contains 3,882 cleaned emails with a stable 3,008-dimensional feature vector. The Random Forest classifier achieved 98.5% accuracy and an F1-score of 0.979 with only 12 misclassifications out of 777 test emails. The browser extension successfully injects real-time verdict banners into Gmail, delivering predictions, confidence scores, and indicator breakdowns without disrupting the user's email workflow.

Chapter 6

Limitations, Recommendations and Conclusion

6.1 Introduction

This chapter presents the limitations of the system, recommendations for future improvement, and a conclusion summarising the work carried out.

6.2 Limitations

6.2.1 Dataset Scope

The system was trained on the SpamAssassin Public Corpus, which contains English-language emails from the early 2000s. Modern phishing tactics may not be well represented, potentially reducing performance on current attacks.

6.2.2 Text-Only Analysis

The classifier analyses email body text only. It does not inspect email headers, attachments, or embedded images, which are common vectors in sophisticated phishing attacks.

6.2.3 Local Deployment Dependency

The browser extension requires the Flask backend to be running locally on `localhost:5000`, which limits usability for non-technical users.

6.2.4 Language Coverage

The system was trained on English text only and does not support Arabic, French, or other languages.

6.2.5 Short Text False Positives

Very short emails can trigger false positive predictions due to insufficient text for the classifier to analyse reliably.

6.3 Recommendations

- **Dataset expansion:** Include more recent phishing datasets such as IWSPA-AP or PhishTank samples.
- **Header analysis:** Add SPF/DKIM validation and sender domain features to detect spoofed-sender attacks.
- **Cloud deployment:** Host the backend on a cloud platform to remove the local server dependency.
- **Multilingual support:** Retrain on multilingual corpora to extend coverage to Arabic and French.
- **Model updates:** Implement periodic retraining using newly reported phishing samples.
- **Deep learning:** Evaluate transformer models such as DistilBERT as a replacement for the current pipeline.

6.4 Conclusion

This project designed and implemented an intelligent phishing email detection system using machine learning, delivered through a browser extension that integrates directly into Gmail.

The pipeline processes raw emails from the SpamAssassin Public Corpus into a 3,008-dimensional feature vector combining TF-IDF representations with eight hand-crafted indicators. Random Forest was selected as the production model with an F1-score of 0.979 and 98.5% test accuracy, misclassifying only 12 out of 777 emails.

The browser extension automatically analyses each email when opened in Gmail and injects a colour-coded verdict banner displaying the predicted

label, confidence score, and suspicious indicators — without requiring the user to leave their inbox.

The limitations identified provide a clear roadmap for future work, with potential extensions including cloud deployment, multilingual support, and deep learning models.

References

[1] Alaa Al-Momani et al. A survey of phishing email filtering techniques. *IEEE Communications Surveys & Tutorials*, 15(4):2070–2090, 2013.

[2] André Bergholz et al. New filtering approaches for phishing email. In *Journal of Computer Security*, volume 18, pages 7–35, 2010.

[3] Ian Fette, Norman Sadeh, and Anthony Tomasic. Learning to detect phishing emails. In *Proceedings of the 16th International Conference on World Wide Web*, pages 649–656, 2007.

[4] Jason Hong. The state of phishing attacks, 2012. Communications of the ACM, 55(1):74–81.

[5] Tyler Moore and Richard Clayton. Examining the impact of website take-down on phishing, 2007. Proceedings of the Anti-Phishing Working Group eCrime Summit.

[6] Finn Toolan and Joe Carthy. Feature selection for spam and phishing detection. In *Proceedings of the eCrime Researchers Summit*, 2010.

[7] Arun Vazhayil et al. Comparative study of the detection of malicious urls using shallow and deep networks. In *International Conference on Computer Communication and Informatics*, 2019.